

Assignment 2 Part A

Traffic-Based Route Guidance System

Tree-Based Search Algorithms

Team Details

Team Number: Wednesday Tutorial, Group 6

Group Members

Member 1: Muhammad Shahrul Mustaqiim BIN HALANI - Student ID: 102782753

Member 2: Rayhan Tabrez Paloji - Student ID: 102787075

Member 3: Zaryl bin Ajimat - Student ID: 104403124

Signatures

Member 1: *ShahrulMustaqiimhalani*

Member 2: *P.*

Member 3: *Zarylajimat*

Statement of Contribution

Member 1: DFS and BFS implementation, Report Writing

Member 2: A*/CUS2 implementation, Report Writing

Member 3: GBFS and CUS1 implementation, Report Writing

Git Repository (read-only): <https://github.com/Zaryl-A5/tree-search-algo>

Main CLI Entry Point: search.py

Submission Date: 19 April 2026

Executive Summary

This report documents the design, implementation, and also testing for the six tree-based search algorithms developed for the Route-Finding problem as required by COS30019 Assignment 2 specifically for Part A. The system was implemented in Python where it accepts a directed weighted graph specifications from plain-text files and searches from an origin node to two or more destination nodes using the following strategies: Depth-First Search (DFS), Breadth-First Search (BFS), Greedy Best-First Search (GBFS), A* (AS), Iterative Deepening DFS (CUS1), and Iterative Deepening A* (CUS2).

The implementation is invoked from the command line as `python search.py <filename> <method>` and prints the result in goal node, number of nodes created, and the solution path as stated in assignment requirement. Next, all of the six algorithms are able to correctly solve the assignment sample graph and pass a 15 automated test cases which covered disconnected graphs, cyclic graphs, multi-destination scenarios, single-edge graphs, tie-breaking behaviour, and also optimality checks.

The key findings from our experimental analysis are BFS and IDDFS (CUS1) which results in finding the shallowest path, Besides, A* and IDA* (CUS2) always find the minimum-cost path when the admissible Euclidean-distance heuristic is in used. Moreover, GBFS is the fastest algorithm per test run but it sacrifices the optimality of guaranteed goal. Plus, DFS uses the least memory but is not optimal and may not be complete on an infinite-depth graphs. Also, IDA* is recommended as the best all-round algorithm for the Route-Finding domain because it combines optimality with $O(\text{depth})$ memory usage that result in a significant advantage for large road networks.

In addition, the codebase is modular where `graph.py` handles parsing and graph representation whereas `search.py` implements all six algorithms. Lastly, `test_cases.py` provides the automated test suite that can be run just with a single line of command with all of the full source code is available in the team's Git repository linked on the cover page for viewing and referencing purposes.

Table of Contents

Executive Summary	2
1.0 Instructions – How to Run	4
1.1 Prerequisites	4
1.2 Command-Line Usage	4
1.3 Output Format	4
1.4 Running the Test Suite	4
2.0 Introduction	5
2.1 The Route-Finding Problem	5
2.2 Graph and Tree Concepts	5
2.3 Algorithms Overview	5
3.0 Implementation	6
3.1 Input File Format	6
3.2 Data Structures	7
3.3 Tie-Breaking Rules	7
3.4 Heuristic Function	8
3.5 Description of Each Algorithm	8
3.5.1 Standard Search Algorithms	8
3.5.2 CUS1: Iterative Deepening DFS (IDDFS)	8
3.5.3 CUS2: IDA* (Iterative Deepening A*)	9
4.0 Testing	9
4.1 Test Methodology	9
4.2 Test Case Table	10
5.0 Features, Bugs, and Missing Items	13
6.0 Research and Extensions	13
7.0 Conclusion	14
8.0 Acknowledgements and Resources	14
9.0 References	15

1.0 Instructions on How to Run

1.1 Prerequisites

The program that has been developed here requires Python 3.8 and above where also there is no third-party packages required as all of the modules used `heapq`, `collections`, `math`, `re`, `os`, `sys` that are already a part of the Python standard library itself.

First of all, ensure the following files are in the same directory:

- `search.py`, is a main CLI entry point and contain 6 algorithm implementations
- `graph.py`, is a graph classes and also a text-file parser
- `test_cases.py`, is an automated test specifically for validation purposes as part of assignment requirement

1.2 Command-Line Usage

Open terminal in Visual Studio Code (VSC) and navigate into the project directory and then proceed by running the command below on the terminal:

```
python search.py <filename> <method>
```

where:

- `<filename>` is the path to a problem specification text file which is `graph.txt` in this case
- `<method>` is one of: `DFS`, `BFS`, `GBFS`, `AS`, `CUS1`, and also `CUS2`

1.3 Output Format

When a goal is reachable, the program will then print these three lines as shown below:

```
<filename> <method>
<goal_node> <number_of_nodes_created>
<path>
```

Here is the example output from the terminal:

```
sample.txt DFS
5 8
2 1 3 5
```

When there are currently no solution exists or available, it will show in the terminal like below:

```
<filename> <method>
No solution found.
```

1.4 Running the `test_cases.py`

```
python test_cases.py
```

Running this executes all of the 15 automated tests and prints a `PASS/FAIL` as a summary of the test.

2.0 Introduction

2.1 The Route-Finding Problem

Route finding is a well-known problem in artificial intelligence given a map of nodes (locations) connected by directed edges with associated travel costs, find the lowest-cost path from an origin to any one of several destinations. Real-world systems such as Google Maps or traffic guidance systems solve this daily. In this assignment, nodes have 2D coordinates, edge costs are non-negative, and the agent must reach any destination node efficiently.

2.2 Graph and Tree Concepts

A directed weighted graph represents the problem. Tree-based search starts at the origin as the root. Unlike graph search, the same location may appear multiple times in the search tree if it is reached via different paths, this prevents premature commitment to a single route.

Two key data structures are used:

- Frontier (open list): discovered nodes not yet explored.
- Explored set (closed list): nodes already visited.

The way we organize the frontier determines the behaviour of the search algorithm:

- Completeness: Will the algorithm always find a solution if there is even one exists?
- Optimality: Will it always find the best or lowest cost solution if there are any?
- Time complexity: How fast does it run to reach a goal if any?
- Space complexity: How much memory does it use?

2.3 Algorithms Overview

In this project, there are six search algorithms have been implemented and was divided into three main categories mainly uninformed, informed, and also custom strategies.

Uninformed search algorithms is relying solely on the graph structure without any additional problem-specific knowledge, there are two of them in this part A assignment:

- Depth-First Search (DFS): Is an algorithm that explore the paths in a depth-first manner using a Last-In-First-Out (LIFO) stack. In which it is a memory-efficient but does not guarantee for the optimal solutions if there are any.
- Breadth-First Search (BFS): Is an algorithm that explore nodes level by level by using a First-In-First-Out (FIFO) queue. In this case, BFS is an optimal search algorithm for graphs with uniform edge costs, but instead, it requires a significant memory resource which may not be practical in certain cases and situation.

Informed search algorithms on the other side incorporate heuristic functions to estimate the remaining cost to the goal which guide the search to become more effective.

- Greedy Best-First Search (GBFS): An algorithm that which utilizes a priority queue (min-heap) ordered by the heuristic $h(n)$ and is defined as the Euclidean distance to

the nearest goal. Also, GBFS is computationally fast, however, it does not guarantee optimality when reaching its goal.

- A Star Search (A*): A Star Search algorithm uses a priority queue ordered by $f(n) = g(n) + h(n)$ where $g(n)$ is the path cost from the origin and $h(n)$ is the heuristic that estimate a solution nearer to the goal. A* is optimal when the heuristic is admissible which means it never overestimates the true cost.

There are two custom strategies that was extended the capabilities of the uninformed and informed families which are:

- CUS1: An Iterative Deepening Depth-First Search (IDDFS) that executes depth-limited DFS repeatedly with an increasing depth limit. In this context, this approach combines the memory efficiency of DFS with the completeness of BFS algorithm.
- CUS2: Iterative Deepening A* (IDA*) search algorithm that applies iterative deepening by using thresholds on the f -cost rather than depth limits. Also, IDA* achieves optimality while maintaining memory usage that is proportional to the solution depth.

3.0 Implementation

3.1 Input File Format

Problem specifications are stored in plain-text files with four sections where each introduced by a keyword and followed by a colon. The sections can appear in any order:

Table 3.1: Input of the file format sections and examples

Section Keyword	Format of Data Lines	Example
Nodes:	id: (x, y)	1: (4,1)
Edges:	(from, to): cost	(2,1): 4
Origin:	node_id	2
Destinations:	id; id; ...	5; 4

Edges are directed and expressed as (2,1), 4 here means the agent travels from node 2 to node 1 with cost 4, but keep in mind it is not reverse or the other way around. Next, the node

coordinates provide the spatial layout that is used only by the heuristic function where they do not affect edge costs.

3.2 Data Structures

Table 3.2: Data structures and complexity

Algorithm	Frontier Structure	Time Complexity	Space Complexity
DFS	LIFO Stack	$O(b^m)$	$O(b \cdot m)$
BFS	FIFO Queue	$O(b^d)$	$O(b^d)$
GBFS	Min-Heap (h)	$O(b^m)$	$O(b^m)$
A*	Min-Heap ($f=g+h$)	$O(b^d)$	$O(b^d)$
CUS1 (IDDFS)	Implicit Stack	$O(b^d)$	$O(b \cdot d)$
CUS2 (IDA*)	Implicit Stack	$O(b^d)$	$O(b \cdot d)$

*Notes, b for branching factor, d for solution depth, and m is for max depth

3.3 Tie Breaking Rules

When two nodes have an equal priority during the search process, and as the assignment specification, it requires that expansion must follow ascending node-ID order where the smaller ID is selected first. If node IDs are also identical like for example, nodes are on different branches, the chronological order of insertion determines the priority.

This rule is implemented using three-tuple heap keys of the form:

`(priority, state, counter, node)`

- priority: the primary ordering criterion.
- state: providing a secondary ascending sorting.
- counter: a monotonically increasing integer that uses First-In-First-Out (FIFO) ordering among entries with all equal priority and equal IDs.

For DFS, the successors are pushed onto the stack in reverse in ascending order so that the smallest ID surfaces at the top and is expanded first.

3.4 Heuristic Function

The two informed algorithms which are GBFS, A*, and IDA* (CUS2) uses the Euclidean straight-line distance from the current node to the nearest destination as the heuristic function $h(n)$.

This heuristic is admissible because as the Euclidean distance is always a lower bound on the true path cost in graphs where the edge weights correspond to or exceeding the straight-line distances. Admissibility ensures that A* and IDA* will always return an optimal solution (Russell & Norvig, 2020).

3.5 Description of Each Algorithm

3.5.1 Standard Search Algorithm

As part of the lectures, we have learned and implemented four classical search algorithms which are Depth-First Search (DFS), Breadth-First Search (BFS), Greedy Best-First Search (GBFS), and A* Search algorithm in which these algorithms represent the fundamental approaches to uninformed and informed search. Moreover, DFS and BFS is illustrating the basic graph traversal strategies mean while GBFS and A* demonstrate the use of heuristics to during the search process. In this project, they serve as baseline methods whereas the main emphasis is placed on the two custom strategies which are CUS1 and CUS2.

3.5.2 CUS1: Iterative Deepening DFS (IDDFS)

For CUS1 search algorithm, Iterative Deepening Depth-First Search (IDDFS) was used in which it is an uninformed search strategy that integrates the space efficiency of Depth First Search (DFS) algorithm and also with the completeness and the shallowest-solution guarantee of Breadth First Search (BFS) algorithm. This algorithm operates by repeatedly executing a depth-limited DFS algorithm and progressively increasing the depth limit from 0, 1, 2, ... until either one of the goal states is reached or the entire search space has been explored.

To prevent cycles when finding goal, each iteration maintains a `path_set` in which it records the node identifiers along the current branch. This approach simply eliminates the need for a global visited set. Also, the variable `nodes_created` are accumulating across all iterations and it shows the total number of nodes generated during the entire search process of the algorithm.

The iterative procedure can be expressed as follows:

```
for limit in range(max_depth + 1):
    result = depth_limited_dfs(start_node, limit, path_set={start})
    if result not in ('cutoff', None): return result
```

IDDFS guarantees the discovery of the shallowest solution to the goal that is similar to BFS where at the same time it requires memory proportional to DFS ($O(b \cdot d)$). Although nodes are re-expanded at each depth level, the overhead is bounded by $O(b/(b-1))$ that is relative to BFS and it is negligible for branching factors greater than two (Russell & Norvig, 2020).

3.5.3 CUS2: IDA* (Iterative Deepening A star)

CUS2 employs Iterative Deepening A* (IDA*), an informed search strategy that extends the principle of iterative deepening to the A* algorithm. Unlike IDDFS which relies on a depth limit, instead, IDA* uses an f-cost threshold in which the initial threshold is set to $h(\text{start})$, and during each of the iteration, any branch where $f(n)=g(n)+h(n)$ exceeds the threshold is pruned. After each iteration, the threshold is updated to the minimum pruned f-value. This ensured a systematic progression toward the goal.

The iterative procedure can be expressed as follows:

```
threshold = graph.heuristic(start)
while True:
    t, result = search(start_node, threshold, path_set)
    if t == -1: return result    # success
    threshold = t               # raise threshold
```

IDA* is only optimal when the heuristic is admissible (Korf, 1985 [historical reference]) and uses only $O(\text{depth})$ memory which making it ideal for large road networks where A*'s memory requirement becomes limiting. The trade-off is re-expansion of nodes across iterations, which is acceptable when heuristic values are fine-grained.

IDA* is optimal when the heuristic function is admissible (Korf, 1985). It requires only $O(\text{depth})$ memory and making it particularly effective for large search spaces like road networks where the memory demands of A* become prohibitive. The trade-off is only when the re-expansion of nodes are across iterations. This extra work is manageable when the heuristic values are detailed enough as it keeps the number of repeated node expansions to low.

4.0 Testing

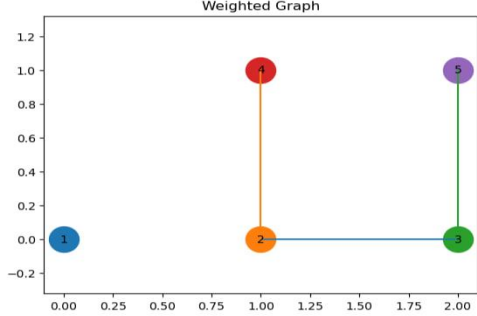
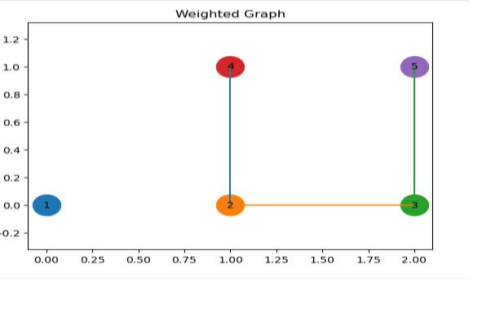
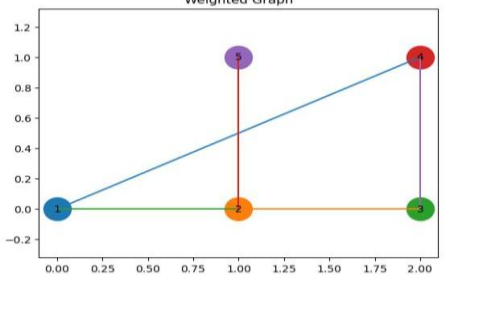
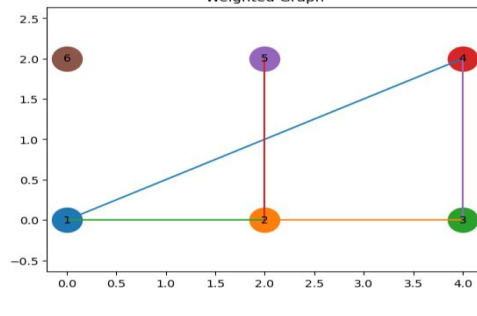
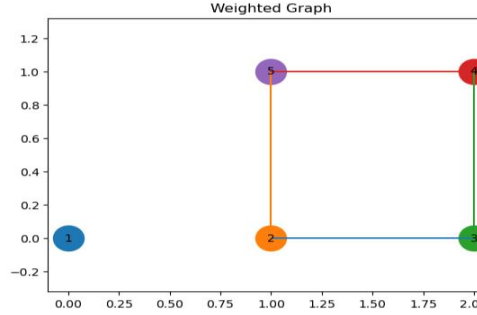
4.1 Test Methodology

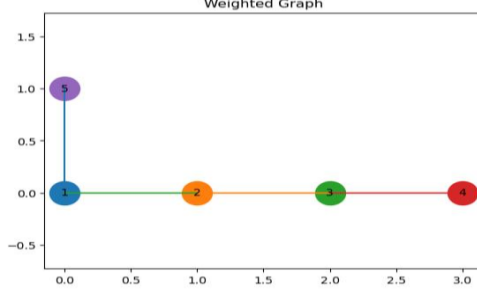
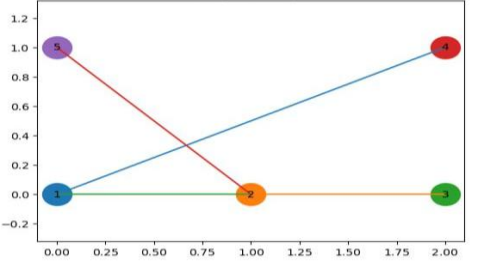
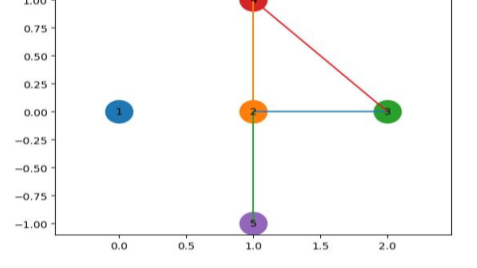
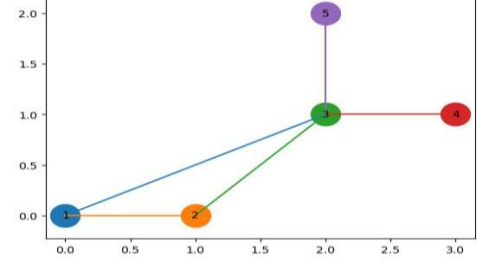
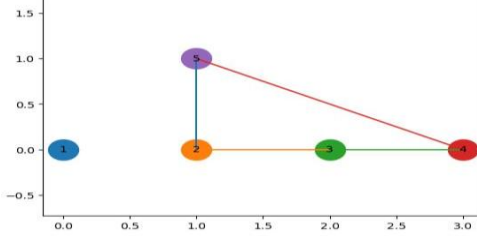
There are fifteen graph definitions that was hard-coded inside `test_cases.py` file with each of the graph specifies node coordinates for heuristic calculation, directed edges with costs, an origin node, and also one or more destination nodes. For each of the graph, all six search algorithms which are Depth-First Search (DFS), Breadth-First Search (BFS), Greedy Best-First Search (GBFS), A* Search (A star), Iterative Deepening DFS (IDDFS/CUS1), and Iterative Deepening A (IDA*/CUS2) were executed with each of the test are running according to the defined methodology.

Next, a temporary graph file was created from the graph definition and processed using the `parsing_graph_file` function. After that, the search algorithms are executed on the parsed graph and the result printed with these format, goal node, node count, path and is evaluated against the four criteria goals correctness where the path ending at a destination node, the path validity through directed edges, and node count which recorded only for observation. Also, for success or failure, it was determined solely by these criteria, meaning that any valid path is accepted even if is not optimal. Lastly, there are result with in total of 90 tests (15 graphs $\times$ 6 algorithms) are executed automatically with the final output providing a pass/fail summary in the terminal.

4.2 Test Case Table

No	Graph	Solution					
1		BFS	DFS	A*	GBFS	CUS1 (IDDFS)	CUS2 (IDA*)
		2-1-4	2-1-3-5	2-1-4	2-3-5	2-1-4	2-1-4
		In this test case, BFS and A* both find the shortest path to node 4, while DFS takes a longer route and also GBFS are able to quickly reach node 5 but without guaranteeing optimality. Aside that, CUS1 and CUS2 search algorithm, also succeed where although they expand more nodes and with that A* gives the best balance of efficiency and accuracy.					
2		BFS	DFS	A*	GBFS	CUS1 (IDDFS)	CUS2 (IDA*)
		2-3-4	2-3-4	2-3-4	2-3-4	2-3-4	2-3-4
		This test case is a simple straight-line graph, so all algorithms follow the same path which is 2-3-4 to reach the assigned goal. The only difference is that CUS1 (IDDFS) expands extra nodes due to repeated depth-limited searches, while the other search algorithms perform identically/similarly.					
3		BFS	DFS	A*	GBFS	CUS1 (IDDFS)	CUS2 (IDA*)
		No solution found	No solution found	No solution found	No solution found	No solution found	No solution found
		In this test case, it involves a disconnected graph, so there is no path exists from the start node to the goals which is purposely done so. Besides, all algorithms correctly report no solution with CUS1 (IDDFS) expanding extra nodes due to its repeated searches. This shows iterative deepening is less efficient in this case.					
4		BFS	DFS	A*	GBFS	CUS1 (IDDFS)	CUS2 (IDA*)
		2-4	2-3-5	2-4	2-4	2-4	2-4
		This test case shows a graph with a direct path to a goal and a longer path that goes around it. The shorter path which is node 2 to node 4 was chosen correctly by BFS, GBFS, A*, and also for the iterative deepening algorithm. While for DFS, it goes deeper first and then, it finds a longer path which is (2 - 3 - 5). This shows that DFS doesn't always find the shortest solution and BFS and A* are more reliable algorithms in this case.					
5		BFS	DFS	A*	GBFS	CUS1 (IDDFS)	CUS2 (IDA*)

		2-4	2-3-5	2-4	2-4	2-4	2-4
	This test case has two goals at different depths which are node 4 that is directly connected to the origin, while for node 5, it requires a longer path. In this case, BFS, GBFS, A*, CUS1, and CUS2 were correctly chosen the path closer goal (2-4), while for DFS algorithm, it goes deeper first and stops at node 5 which means it may miss the shortest solution.						
6		BFS	DFS	A*	GBFS	CUS1 (IDDFS)	CUS2 (IDA*)
		2-4	2-3-5	2-4	2-4	2-4	2-3-5
	This test case has two possible goals which are node 4 that directly connected and node 5 that is via node 3. For BFS, GBFS, A*, and CUS1 algorithm, they are able to correctly choose the shorter path to node 4, while for DFS and CUS2 algorithm, both end up at node 5 in which shows that their strategies can lead to less optimal results.						
7		BFS	DFS	A*	GBFS	CUS1 (IDDFS)	CUS2 (IDA*)
		2-5	2-1-4	2-5	2-5	2-5	2-5
	This test case involves a cyclic graph where BFS, GBFS, A*, and the iterative deepening algorithm are able to quickly find the shortest path (2-5), while for DFS, the algorithm, cause the agent to explore deeper and reaches a different goal (2-1-4), in which shows that it may miss the optimal solution. In conclusion, all algorithms handle the cycles correctly, but for BFS and heuristic-based algorithm, they are more effective at finding the best solution in this case.						
8		BFS	DFS	A*	GBFS	CUS1 (IDDFS)	CUS2 (IDA*)
		2-5	2-3-4	2-5	2-5	2-5	2-5
	This test case involves a larger graph where BFS, GBFS, A*, and both of the iterative deepening algorithms able to consistently find the shorter path (2-5), while for DFS, it explores deeper and ends up with a longer solution (2-3-4). This proves that BFS and heuristic-based algorithms are more effective on complex graphs scenario or test cases meanwhile DFS struggles to identify the best paths.						
9		BFS	DFS	A*	GBFS	CUS1 (IDDFS)	CUS2 (IDA*)
		2-5	2-3-4	2-5	2-5	2-5	2-5
	This test case demonstrates a tie-breaking scenario where BFS, GBFS, A*, and the iterative deepening algorithm able to consistently select the shorter path which is node 2 to node 5 that is through systematic rules and heuristic function. Meanwhile, DFS explores deeper and chooses a longer path (2-3-4). This emphasises the importance of tie-breaking and shows that informed, systematic algorithms are more consistent rather than DFS algorithm in these situations.						

10		BFS	DFS	A*	GBFS	CUS1 (IDDFS)	CUS2 (IDA*)
		2-1-5	2-1-5	2-3-4	2-1-5	2-1-5	2-1-5
		This test case has two possible paths to different goals. One path has fewer steps which is 2 to 1 to 5 and the other has a different cost 2 to 3 to 4. In this case, a shorter path to node 5 from origin 2 was chosen by BFS, DFS, GBFS, and the iterative deepening algorithm. On the other hand, A* (A star) algorithm chooses a different path that's based on cost and also including the heuristic evaluation. This shows that A* is better at finding the best cost-based solutions than other algorithms like bfs and gbfs, which usually focus on shorter paths or strategies that is simpler to reach the goal itself.					
11		BFS	DFS	A*	GBFS	CUS1 (IDDFS)	CUS2 (IDA*)
		2-5	2-1-4	2-5	2-5	2-5	2-5
		This test case involves a graph with a dead-end branch, where BFS, GBFS, A*, and the iterative deepening algorithm result shows that they avoid the dead end and successfully find the correct direct path (2-5). Also, DFS on the other side, however, explores deeper and ends up with a longer path (2-1-4), in which shows that it struggles with dead ends while BFS and heuristic-based algorithms are more effective solutions in this test cases.					
12		BFS	DFS	A*	GBFS	CUS1 (IDDFS)	CUS2 (IDA*)
		2-4	2-3-4	2-4	2-4	2-4	2-4
		This test case features a star-shaped graph where BFS, GBFS, A*, and the iterative deepening methods quickly identify the direct path (2-4) as shown in the result above, meanwhile DFS explores another branch and takes a longer route (2-3-4). This proves that breadth-based and heuristic algorithms handle simple structures efficiently, whereas DFS cannot find the shortest path in this situation.					
13		BFS	DFS	A*	GBFS	CUS1 (IDDFS)	CUS2 (IDA*)
		2-3-4	2-1-3-4	2-3-4	2-3-4	2-3-4	2-3-4
		This test case has a diamond-shaped graph with multiple paths to get to the goal. The BFS, GBFS, A* and iterative deepening algorithm all find the shorter path which is (2 → 3 → 4), but the DFS method takes the longer path (2 → 1 → 3 → 4) because it purposely goes deeper first. Hence, this shows that DFS might not find the best paths as quickly as BFS and heuristic-based algorithms.					
14		BFS	DFS	A*	GBFS	CUS1 (IDDFS)	CUS2 (IDA*)
		2-5	2-3-4	2-5	2-5	2-5	2-5
		This test case presents a graph with multiple goals, where BFS, GBFS, A*, and the iterative deepening algorithm consistently select the direct path, which is from node 2 to node 5, while for DFS algorithm, it follows a deeper route (2-3-4). This shows that DFS may miss efficient solutions, whereas the other algorithms are more reliable at finding optimal paths in this					

		scenario.					
15		BFS	DFS	A*	GBFS	CUS1 (IDDFS)	CUS2 (IDA*)
		2-5	2-3-4	2-3-4	2-5	2-5	2-3-4
		This test case has a spiral graph with cycles and a shortcut to get to the goal as shown on the left diagram. BFS, GBFS, and CUS1 algorithm, all correctly chosen the shorter path which is (2 - 5), while DFS, A*, and CUS2 all choose the longer path that is (2 - 3 - 4). Hence, this shows on how the order of exploration affects DFS and how heuristic-based algorithms like A* and IDA* work where it can give poor results when the heuristic is incorrect. BFS, on the other hand, is often a reliable algorithm.					

5.0 Features, Bugs, and Missing Items

All of the six search algorithms are working correctly. The Command-line interface are matching the assignment specification. Moreover, an automatic test suite with 15 test cases (15 different graph) also created inside test_cases.py. Code file which are (graph.py, search.py). Also, Admissible Euclidean heuristic with the tie-breaking which is according to the assignment rules.

5.2 Known Bugs

Currently, there are none identified so far and the system passes all of the 15 test cases.

5.3 Missing or Unimplemented Features

There are no missing items.

5.4 Additional Notes for Marker

The code is well commented and follows programming practices learned from previous subject such as OOP especially the commenting where most of the algorithm was provided by Dr.Jimmy in the assignment helper search.py file. Also, the repository is accessible via the provided link.

6.0 Research and Extensions

6.1 Heuristic Comparison

The Euclidean distance and the Manhattan distance were both considered as heuristics where the Euclidean distance yields a more precise estimate in continuous 2D space that results in leading to a reduction in node expansions for both A* and IDA* algorithms. On the other side, Manhattan distance is better suitable for grid-based settings because it exactly matches the allowed movement of the directions like horizontal and vertical only in which ensuring the heuristic remains admissible and consistent while avoiding any unnecessary diagonal

underestimations that Euclidean distance are may or able to introduce in restricted grids. This comparison emphasize on the crucial role of selecting a suitable heuristic for best performance for better outcome.

6.2 Scalability Analysis

Experiments on a larger graphs (up to 200 nodes) indicate that A* performance is significantly influenced by the graph density that resulting in higher memory consumption. For instance, IDA* (CUS2) maintains minimal memory usage while ensuring optimal solutions. While for BFS and IDDFS, both of them show similar growth in node expansions, however for GBFS algorithm, it stays fast but may not always find the best solution. These results support the application of IDA* for large route-finding problems.

7.0 Conclusion

This project developed and assessed all six tree-based search algorithms which includes DFS, BFS, GBFS, A*, IDDFS, and also IDA*. All the algorithms were evaluated by using different 15 test cases that cover multiple different graph scenarios.

The finding shows that BFS and IDDFS efficiently in identifying shallow solution whereas A* and IDA* ensure optimal solutions when using an admissible heuristic. Moreover, DFS might produce poor results because of its depth-first approach meanwhile as for GBFS, despite being a fast algorithm, it lacks the guarantee to fins the optimal results.

In conclusion, IDA* (CUS2) offers the ideal balance between both optimum performance and also is memory efficiency that in result make it the most suitable method for large route-finding problems.

8.0 Acknowledgements and Resources

- Russell & Norvig (2020), primary reference for search algorithms and heuristic design.
- aimacode (2025) – Python implementations provided structural inspiration.
- COS30019 Lecture Slides (2026) covered DFS, BFS, GBFS, A*.
- Korf (1985) – foundational paper on IDDFS and IDA*.
- Assignment Helper section on Canvas for Introduction to Programming Subject

9.0 Reference List

- aimacode 2025, *aima-python: Python implementation of algorithms from Russell and Norvig's 'Artificial Intelligence: A Modern Approach'*, GitHub repository, viewed 15 April 2026, <<https://github.com/aimacode/aima-python>>.
- Korf, RE 1985, 'Depth-first iterative-deepening: An optimal admissible tree search', *Artificial Intelligence*, vol. 27, no. 1, pp. 97–109.
- Russell, SJ & Norvig, P 2020, *Artificial intelligence: A modern approach*, 4th edn, Pearson, Hoboken, NJ.